Scenario: "This React App is Slow. How Do You Fix It?"

3 min read Updated 24 Mar 2026 Topic 43 of 62

Bookmark Mark done

10.1 - Scenario: "This React App is Slow. How Do You Fix It?"

🎯 Why This Question Exists

This is the most common senior-level React interview question. It tests whether you can diagnose and fix performance problems systematically, not just recite optimization buzzwords.

The Framework for Answering

Never jump to solutions. Walk through a structured diagnosis.

Step 1: Identify the Symptom

Ask the interviewer clarifying questions:

- "Is it slow on initial load, or slow during interactions?"
- "Is it a specific page or the whole app?"
- "Are there any large lists or heavy computations?"
- "Is it slow on all devices or just low-end ones?"

Step 2: Measure

Before touching any code, profile the app:

- **React DevTools Profiler** to find which components re-render unnecessarily and how long they take.
- **Chrome Performance tab** to find long JavaScript tasks blocking the main thread.
- **Lighthouse** to measure Core Web Vitals (LCP, INP, CLS).
- **Network tab** to check bundle size, image sizes, and API response times.
- **Bundle analyzer** to see what's inside the JavaScript bundle.

Step 3: Diagnose and Fix

Based on what the profiling reveals, apply the appropriate fix:

Symptom	Likely Cause	Fix
Slow initial load, large JS bundle	Too much code loaded upfront	Code splitting with React.la:
Slow initial load, large images	Unoptimized images	WebP/AVIF format, lazy loac
Slow initial load, many API calls	Waterfall requests on mount	Combine APIs, use SSR/SSG

interactions, entire page re-renders	State too high in the tree	Move state down, split com
Slow interactions, child components re-render with same props	Missing memoization	React.memo on expensive c
Slow interactions, huge list rendering	Thousands of DOM nodes	Virtualize with react-windo
Slow typing in search input	API call on every keystroke	Debounce the input (useDe
Layout shifting on load	Missing image dimensions, late-loaded content	Set explicit width/height, us

Step 4: Verify

After applying fixes, re-run profiling to confirm the improvement. Measure before and after.

"My approach would be to first understand the symptom: is it a slow initial load or slow interactions? Then I'd profile the app. For initial load, I'd check bundle size with a bundle analyzer and look for opportunities to code-split, lazy-load routes, and replace heavy dependencies. For interaction slowness, I'd use the React Profiler to find components that re-render unnecessarily. My first fix would be restructuring: moving state down and splitting components. If that's not enough, I'd apply React.memo to expensive children. For long lists, I'd virtualize. For search inputs, I'd debounce. And I'd always measure before and after to confirm the improvement."

QUICK CHECK — no marks, just for you.

1. An interviewer says "this React app is slow — how would you fix it?". Per the lesson's framework, why does Step 1 consist of asking clarifying questions like "Is it slow on initial load, or slow during interactions?" before naming any fix?

- ○ Because slow initial load and slow interactions have completely different root causes, so the symptom determines which fixes are even relevant
- ○ Because initial-load problems can always be ignored — only interaction slowness is measurable with the React DevTools Profiler
- ○ Because the interviewer is testing whether you know that React.memo is the universal first fix for any kind of slowness
- ○ Because clarifying questions buy time while you recall the list of optimization APIs, which is the actual thing being graded

Check answer

2. Profiling shows the app freezes for long stretches during use. According to the lesson's Step 2 tooling, which tool is the one specifically meant for finding long JavaScript tasks blocking the main thread?

- Lighthouse
- Network tab
- Bundle analyzer

Check answer

3. The React Profiler shows child components re-rendering on every interaction. Per the ordering in the lesson's sample answer, what should you reach for FIRST?

- Restructure: move state down and split components, and only apply React.memo to expensive children if that isn't enough
- Wrap every child in React.memo immediately, since memoization is the cheapest and most reliable fix
- Virtualize the component tree with react-window, since re-renders are always a DOM-node-count problem
- Debounce the interaction handlers, since re-renders on interaction are always caused by too-frequent events

Check answer

‹ Previous
Mocking API Calls, Hooks & Context

Next ›
Scenario: "Design the Component Architecture for This Feature"

LEARN MORE

ARTICLES & DOCS

memo — API reference
react.dev

useMemo — API reference
react.dev

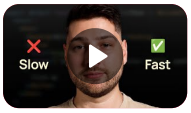
VIDEOS



React performance: fix a slow app
YouTube



React performance optimization patterns
YouTube



Best ways to optimize modern React
YouTube

Hand-picked references for this lesson.